

Multi-marker constellation (Marker Constellation Design)

PHỤ THUỘC A/03-pnp-va-uoc-luong-tu-the (PnP hợp nhất trên toàn bộ tập góc, và điều kiện để nó tốt hơn trung bình pose); A/04-pose-ambiguity-target-phang (vì sao phải phá tính đồng phẳng); A/05-lan-truyen-sai-so-pixel-sang-pose (quan hệ $\delta\theta \approx \delta x/B$ — toàn bộ chương này là việc hiện thực hoá quan hệ ấy bằng nhôm và vít).

TÓM TẮT

Đây là chương có đòn bẩy lớn nhất trong cả cây, và điều làm nó có đòn bẩy lớn là nó không nói về phần mềm. Ba kết quả của Phần A — sai số góc tỉ lệ nghịch với baseline, ambiguity khử được bằng tính không đồng phẳng, và PnP hợp nhất thắng trung bình pose — đều trở về cùng một hành động: *thiết kế hình học của tấm marker cho đúng*. Chương này biến ba kết quả ấy thành một quy trình thiết kế gồm năm quyết định: baseline bao nhiêu, bố trí không đồng phẳng thế nào, dùng mấy cỡ tag, đo lại constellation ra sao, và trọng số hoá thế nào. Trong năm quyết định ấy, quyết định thứ tư — *đo lại constellation bằng bundle adjustment thay vì tin bản vẽ CAD* — là quyết định mà tôi cho là ranh giới giữa đạt và không đạt 5 mm. Nếu chỉ nhớ một điều: *đừng tin CAD, hãy tự đo* — vì sai số lắp ráp của tấm marker trở thành sàn sai số của cả hệ thống, và nó là sai số hệ thống nên không trung bình mất đi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **constellation** — tập các marker gắn cứng với nhau, cùng với toạ độ 3D của mọi góc của chúng trong một hệ toạ độ chung — hệ dock. Constellation không phải một tập tag; nó là một tập tag *cộng với* một mô hình hình học, và chất lượng của mô hình ấy quan trọng ngang chất lượng của các tag.
- **baseline** B — khoảng cách giữa hai marker xa nhau nhất theo một phương. Đáng chú ý: baseline là một đại lượng *có hướng* — một constellation trải rộng theo chiều ngang nhưng hẹp theo chiều dọc cho sai số yaw tốt và sai số pitch tệ.
- **độ lệch khỏi mặt phẳng** — khoảng cách lớn nhất từ một điểm điều khiển tới mặt phẳng khớp tốt nhất qua toàn bộ tập điểm. Đây là đại lượng định lượng cho “không đồng phẳng tới mức nào”, và nó là con số phải đưa vào bản vẽ.
- **nested multi-scale** — bố trí gồm nhiều cỡ tag lồng nhau, sao cho ở mọi khoảng cách trong dải tiếp cận luôn có ít nhất một tag nằm trong dải kích thước ảnh tốt.
- **bundle adjustment (BA)** — bài toán tối ưu đồng thời cả pose của nhiều khung nhìn lẫn toạ độ 3D của các điểm, cực tiểu sai số tái chiếu trên toàn bộ [1]. Trong chương này nó được dùng theo một chiều đặc biệt: giữ tag làm ẩn và dùng nhiều ảnh để *đo* constellation.
- **tự đo gauge** — trong BA, bài toán có các hướng không xác định được từ dữ liệu — vị trí và hướng của toàn bộ hệ toạ độ, và với ảnh đơn thuần thì cả tỉ lệ. Phải cố định chúng bằng quy ước, và cách cố định ảnh hưởng tới cách đọc hiệp phương sai kết quả.
- **chi-square gating** — loại bỏ phép đo có phần dư chuẩn hoá vượt một phân vị của phân bố χ^2 . Nó có cơ sở thống kê chứ không phải một ngưỡng tuỳ ý — nhưng nó đòi hiệp phương sai đầu vào đúng.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có ngân sách cho bốn tag. Phương án A: bốn tag 100 mm xếp thành hình vuông cạnh 400 mm trên mặt phẳng dock. Phương án B: hai tag 100 mm cách nhau 400 mm, mỗi tag nghiêng 20° về hai phía ngược nhau. Phương án nào tốt hơn, và vì sao chênh lệch lớn hơn nhiều so với chênh lệch số tag?
- Q2. Bản vẽ CAD nói hai tag cách nhau đúng 400,0 mm. Sau khi gia công và lắp, khoảng cách thật là 398,6 mm. Sai lệch 1,4 mm ấy làm hỏng ngân sách 5 mm tới mức nào — và vì sao nó không hiện ra ở bất kỳ chỉ số nào trong pipeline?
- Q3. Robot tiếp cận từ 3 m xuống 0,4 m. Với một cỡ tag duy nhất, kích thước trên ảnh thay đổi 7,5 lần. Vì sao điều đó là một vấn đề ở *cả hai* đầu của dải, và bao nhiêu cỡ tag thì đủ?
- Q4. Một tag trong constellation bị bắn nên góc của nó lệch 2 px. Trong PnP hợp nhất, sai lệch ấy lan sang pose như thế nào, và cơ chế nào phát hiện được nó trước khi nó phá hỏng kết quả?
- Q5. Có một phép đo bạn làm được mà không cần bất kỳ thiết bị đo lường nào ngoài chính camera của robot, và nó cho độ chính xác constellation tốt hơn thước cặp. Phép đo ấy là gì, và vì sao nó lại chính xác hơn?

1.1 Đặt vấn đề

Phần A kết thúc với ba kết luận, và cả ba đều trở về cùng một nơi.

Kết luận thứ nhất, từ A/05: sai số góc từ một constellation baseline B là $\delta\theta \approx Z\sigma/(fB)$, giảm tuyến tính theo B . Kết luận thứ hai, từ A/04: target phẳng luôn có hai nghiệm, và cách khử triệt để duy nhất là phá tính đồng phẳng. Kết luận thứ ba, từ A/03: PnP hợp nhất trên toàn bộ tập góc thăng trung bình pose, với điều kiện toạ độ 3D của các tag đúng.

Ba kết luận ấy có một điểm chung đáng chú ý: *không cái nào nói về phần mềm*. Chúng nói về việc đặt các tấm nhôm ở đâu, nghiêng bao nhiêu, và đo chúng cẩn thận tới mức nào. Đây là chương biến chúng thành một bản vẽ.

Bài toán gốc thì cũ hơn robot học. Ngành trắc lượng ảnh đã giải nó từ lâu dưới tên khác: khi đo một công trình bằng ảnh, người ta đặt các mốc mục tiêu (targets) quanh đối tượng và *đo lại vị trí của chính các mốc ấy* bằng cách chụp nhiều ảnh rồi chạy bundle adjustment, thay vì tin vào việc đặt mốc chính xác [1]. Lý do rất thực dụng: đặt mốc chính xác tới phần mười milimét thì đắt, còn đo lại chúng bằng ảnh thì rẻ và chính xác hơn.

Điều làm chương này đáng viết cho người làm docking — thay vì chỉ trở sang tài liệu trắc lượng ảnh — là hai chỗ mà bài toán của ta khác.

Chỗ khác thứ nhất: ta có một *ràng buộc vận hành* mà trắc lượng ảnh không có. Robot tiếp cận theo một đường xác định, từ xa tới gần, và tag phải nhìn thấy được trên toàn bộ đường ấy. Điều này loại bỏ một số bố trí tối ưu về mặt hình học nhưng không dùng được vì tag bị che khuất hoặc quá nhỏ ở một đoạn nào đó. Mục 6 bàn chuyện này.

Chỗ khác thứ hai, và nó quan trọng hơn: ta có một *hướng ưu tiên*. Trắc lượng ảnh thường muốn độ chính xác đều theo mọi hướng; docking thì không. Robot cập vào theo trục z , nên sai lệch dọc trục ít quan trọng hơn sai lệch ngang và sai lệch góc yaw — một robot lệch 3 mm dọc trục thì chỉ vào hơi nông hoặc hơi sâu, còn lệch 1° yaw thì có thể trượt hẳn. Sự bất đối xứng ấy nên được phản ánh trong thiết kế constellation, và mục 3 nói cụ thể.

Cách hiển nhiên nhất — và cách gần như mọi dự án bắt đầu — là dán một tag lớn lên giữa dock. Nó đơn giản, nó hoạt động, và nó trần ở quanh một, hai độ vì lý do đã phân tích ở A/05. Cách hiển nhiên thứ hai là dán thêm tag cho chắc, thường là bốn tag ở bốn góc một tấm bảng. Cách này tốt hơn thật — nó tăng baseline — nhưng nó bỏ lỡ một nửa lợi ích vì bốn tag ấy vẫn đồng

phẳng nên ambiguity vẫn còn nguyên. Chương này lập luận rằng nửa bị bỏ lỡ ấy thường lớn hơn nửa đã lấy được.

TRỰC GIÁC

Hình dung việc đọc hướng của một vật bằng cách nhìn hai cái mốc gắn trên nó.

Hai mốc gần nhau thì khó: quay vật đi một chút, hai mốc dịch chuyển gần như nhau, và bạn phải đo được cái chênh lệch nhỏ giữa hai dịch chuyển gần bằng nhau. Hai mốc xa nhau thì dễ: cùng một góc quay làm chúng dịch ngược chiều nhau một khoảng lớn.

Đó là baseline, và nó giải thích tại sao trải rộng thì tốt. Nhưng có một chuyện thứ hai, và nó không nằm trong hình ảnh trên.

Giả sử hai mốc ấy nằm trên cùng một mặt phẳng với vật, phẳng lì. Bây giờ có một câu hỏi mà việc trải rộng không trả lời được: mặt phẳng ấy đang nghiêng về phía nào? Nghiêng lên trên hay nghiêng xuống dưới? Từ một cái bóng phẳng, hai khả năng ấy trông giống nhau — đây chính là bài toán hai nghiệm.

Cách chữa không phải là đặt mốc xa hơn nữa. Cách chữa là *dựng một cái mốc nhô ra khỏi mặt phẳng*. Ngay khi có một điểm không nằm trên mặt phẳng ấy, hai khả năng không còn giống nhau nữa: cái mốc nhô ra sẽ che khuất hoặc lộ ra tùy theo mặt phẳng nghiêng về phía nào.

Hai chuyện ấy độc lập nhau, và đó là điều quan trọng nhất của chương. Trải rộng chữa bài toán *độ nhạy*. Nhô ra chữa bài toán *hai nghiệm*. Bạn cần cả hai, và may mắn là một thiết kế tốt cho cả hai cùng lúc: đặt tag xa nhau *và* nghiêng chúng đi.

Còn một chuyện thứ ba, và nó là chuyện dễ quên nhất. Mọi lập luận trên đều giả định bạn *biết* hai cái mốc cách nhau bao xa. Nếu bạn nghĩ chúng cách nhau 400 mm mà thực ra là 398,6 mm, thì mọi phép đo góc của bạn sai theo một tỉ lệ cố định, và không có gì trong dữ liệu báo cho bạn biết.

1.2 Nguyên tắc một: PnP hợp nhất, và điều nó đòi hỏi

Điểm xuất phát về mặt phần mềm, và nó ngắn vì A/03 mục 5 đã dẫn.

Với M tag nhìn thấy, chạy *một* bài toán tối ưu trên toàn bộ $4M$ góc, với tọa độ 3D của mọi góc biểu diễn trong hệ dock. Không trung bình pose. Lý do đã rõ: trung bình pose vứt bỏ ràng buộc rằng các tag gắn cứng với nhau, mà chính ràng buộc ấy tạo ra cơ chế baseline.

Điều đáng nhấn mạnh ở đây là *điều kiện* của lời khuyên ấy, vì nó là chỗ nối sang toàn bộ phần còn lại của chương. PnP hợp nhất tin vào mô hình constellation và ép một pose duy nhất giải thích mọi góc. Nếu mô hình sai, bộ giải phải thỏa hiệp, và nó phân bổ sai lệch vào pose theo cách không kiểm soát được. Nói cách khác: *PnP hợp nhất chuyển sai số mô hình constellation thẳng vào sai số pose*.

Điều này có một hệ quả mà tôi cho là quan trọng và ít được nói: *quyết định dùng PnP hợp nhất và quyết định đo constellation bằng BA là một cặp không tách rời được*. Dùng cái thứ nhất mà bỏ cái thứ hai thì bạn đã đổi một nguồn sai số (thiếu thông tin hình học) lấy một nguồn khác (mô hình sai), và nguồn thứ hai có thể lớn hơn. Đây là lý do mục 5 nằm trong chương này chứ không nằm ở B/12 cùng với các loại hiệu chuẩn khác.

May mắn là có một chỉ báo miễn phí, đã nêu ở A/03: so sai số tái chiếu của bài toán hợp nhất với sai số tái chiếu khi giải từng tag riêng. Nếu hợp nhất lớn hơn hẳn, mô hình constellation sai. Đây là phép thử nên chạy định kỳ chứ không chỉ một lần lúc lắp đặt — tám marker bị va chạm, vít lỏng, nhiệt độ thay đổi.

1.3 Nguyên tắc hai: baseline có hướng

A/05 cho $\delta\theta \approx \delta x/B$ và kết luận “tăng B ”. Mục này làm rõ một chi tiết mà công thức vô hướng ấy giấu đi: *baseline là một đại lượng có hướng, và ba thành phần xoay cần ba baseline khác nhau.*

Đặt hệ toạ độ dock với z hướng ra ngoài (dọc trực tiếp cận), x ngang, y đứng. Ba thành phần xoay:

Yaw (quay quanh y , trục đứng) — đây là thành phần quan trọng nhất cho docking vì nó quyết định robot có vào thẳng trục không. Nó được xác định bởi baseline theo *chiều ngang x* : hai tag cách nhau theo x dịch chuyển ngược chiều khi yaw đổi.

Pitch (quay quanh x , trục ngang) — quan trọng ít hơn với robot di động trên sàn phẳng nhưng vẫn cần cho việc ước lượng đúng các thành phần khác. Được xác định bởi baseline theo *chiều đứng y* .

Roll (quay quanh z , trục quang) — đây là thành phần *dễ nhất*, như A/05 mục 5 đã chỉ ra, vì nó là xoay trong mặt phẳng ảnh và được đo qua cơ chế vị trí. Nó gần như luôn đạt mà không cần thiết kể gì đặc biệt.

Hệ quả thiết kế trực tiếp và nó khác với “trải rộng đều”: *ưu tiên baseline ngang*. Một constellation rộng 500 mm và cao 150 mm cho yaw tốt hơn nhiều so với một constellation vuông 300×300 mm cùng diện tích, và với bài toán docking thì yaw là thành phần đáng tiền nhất.

Nhưng đừng bỏ hẳn baseline đứng, vì một lý do không hiển nhiên: nếu constellation gần như một đường thẳng ngang, thì ma trận thông tin gần suy biến theo hướng pitch, và một ma trận gần suy biến gây hại nhiều hơn chỉ là “pitch kém” — nó làm toàn bộ bài toán kém điều kiện, và sai số theo hướng yếu rò rỉ sang các hướng khác qua các phần tử ngoài đường chéo của $\mathcal{I}^{-1}$. Tỉ lệ mà tôi khuyến nghị là baseline đứng không dưới khoảng một phần ba baseline ngang (*đây là khuyến nghị của tôi dựa trên cơ chế kể trên, không phải kết quả tối ưu hoá đã kiểm chứng — hãy kiểm bằng Monte Carlo ở C/16*).

1.4 Nguyên tắc ba: phá tính đồng phẳng

A/04 kết luận rằng đây là cách duy nhất khử triệt để ambiguity. Mục này nói cụ thể làm thế nào, và bao nhiêu là đủ.

A. Ba cách thi công

Nêm nghiêng dưới mỗi tag. Giữ mọi tag trên cùng một tấm bảng nhưng gắn mỗi tag lên một nêm nghiêng $15\text{--}30^\circ$, với các hướng nghiêng khác nhau. Đây là cách tôi khuyến nghị mặc định vì nó cho *hai* lợi ích cùng lúc: nó phá đồng phẳng (khử ambiguity ở tầng constellation), và nó làm mỗi tag riêng lẻ có góc nghiêng khác không so với trục quang, nên chính tag ấy cũng thoát khỏi vùng $\sin\theta \rightarrow 0$ của A/05 mục 5. Chi phí gia công gần như bằng không.

Tag ở các độ sâu khác nhau. Một số tag trên mặt phẳng dock, một số trên các gờ nhô ra 50–100 mm. Cho baseline theo chiều sâu, thứ mà cách một không cho, và điều đó cải thiện việc ước lượng khoảng cách. Thi công dễ nếu dock vốn đã có cấu trúc ba chiều.

Target dạng góc. Hai hoặc ba mặt phẳng vuông góc nhau, mỗi mặt mang tag. Mạnh nhất về hình học nhưng chiếm không gian và — nhược điểm thật — dễ bị che khuất một phần khi robot tới rất gần, đúng lúc cần chính xác nhất.

B. Bao nhiêu là đủ?

Đây là câu hỏi định lượng, và câu trả lời trung thực là *tôi không có công thức*. Nhưng có một cách suy nghĩ đúng về nó, và nó dẫn tới một phép đo cụ thể.

Cơ chế khử ambiguity là: điểm nằm ngoài mặt phẳng bị phép lật đưa tới vị trí khác hẳn, nên nghiệm thứ hai có sai số tái chiếu lớn hơn hẳn. Để cơ chế ấy hoạt động, chênh lệch sai số tái chiếu do phép lật gây ra phải *vượt hẳn nhiều*. Nghĩa là điều kiện có dạng

$$(\text{độ lệch khỏi mặt phẳng}) \times (\text{hệ số hình học}) \gg \sigma,$$

và hệ số hình học phụ thuộc vào khoảng cách, tiêu cự, và góc nhìn theo một cách mà tôi không dẫn ra được ở dạng đóng.

Cách đúng để có con số cho hệ của bạn là mô phỏng, và mini project của A/04 chính là mô phỏng ấy: quét độ lệch khỏi mặt phẳng và tìm chỗ tỉ số ambiguity ρ bắt đầu lớn ổn định ở mọi tư thế. Đó là ngưỡng của bạn.

Khuyến nghị mặc định của tôi để bắt đầu, trước khi có kết quả mô phỏng: *độ lệch khỏi mặt phẳng ít nhất bằng khoảng 20% kích thước tag*, tức với tag 100 mm thì ít nhất 20 mm. Với nêm nghiêng 20° dưới một tag 100 mm, độ lệch giữa hai mép tag là $100 \times \sin 20^\circ = 34$ mm — thoải mái trên ngưỡng ấy. (Con số 20% là khuyến nghị của tôi dựa trên việc nó cho chênh lệch reprojection vài pixel ở cấu hình điển hình; nó chưa qua của kiểm chứng nào và đã được ghi vào 99-chua-biet.md.)

CẦN NHỚ

Nêm nghiêng là chi tiết gia công có tỉ lệ lợi ích trên chi phí cao nhất trong toàn bộ hệ thống docking.

Nó giải quyết *hai* bài toán độc lập bằng một chi tiết: ambiguity của target phẳng (A/04), và độ nhảy góc bằng không ở fronto-parallel (A/05 mục 5).

Nó tồn: một khối nhôm vát góc dưới mỗi tag, cộng bốn con vít.

Nó phải được đưa vào bản vẽ *trước khi* gia công. Thêm nó sau khi đã sản xuất một trăm cái dock thì tốn một trăm lần chi phí ấy — và đó là lý do chương này nên được đọc trước khi thiết kế cơ khí, không phải sau khi phần mềm đã chạy và không đạt.

1.5 Nguyên tắc bốn: đo constellation bằng BA, đừng tin CAD

Đây là mục mà tôi cho là quan trọng nhất trong cả Phần B, và nó trả lời Q2 cùng Q5.

A. Vì sao CAD không đủ

Bản vẽ nói hai tag cách nhau 400,0 mm. Thực tế thì: tấm nhôm được cắt với dung sai, các lỗ vít được khoan với dung sai, tag được in với sai số tỉ lệ của máy in, tờ in được dán với sai lệch vị trí và có thể hơi lệch góc, và toàn bộ cụm được lắp với dung sai. Cộng lại, sai lệch cỡ một tới vài milimét là hoàn toàn bình thường cho một quy trình gia công thông thường.

Bây giờ hỏi: 1,4 mm sai lệch làm gì với ngân sách 5 mm? Hai câu trả lời, và câu thứ hai mới là câu đáng sợ.

Câu thứ nhất, về độ lớn: nếu constellation sai 1,4 mm trên baseline 400 mm, thì tỉ lệ sai là $1,4/400 = 0,35\%$. Sai số góc tương ứng cỡ $\arctan(1,4/400) = 0,2^\circ$ — tức 40% *ngân sách góc* tiêu vào một dòng duy nhất, và là dòng mà không thuật toán nào chạm tới được.

Câu thứ hai, về tính chất: sai lệch này là *hệ thống*. Nó không đổi giữa các khung, không đổi khi lấy trung bình, không đổi khi cải thiện detector. Theo phân loại của A/05 mục 8, nó cộng thẳng

chứ không cộng bình phương. Và — đây là phần trả lời nửa sau của Q2 — nó *không hiện ra ở bất kỳ chỉ số nào* trong pipeline khi chỉ nhìn một cấu hình: PnP hợp nhất sẽ tìm ra pose tốt nhất giải thích được dữ liệu dưới mô hình sai, sai số tái chiếu tăng lên một chút nhưng vẫn nhỏ, và hiệp phương sai báo về vẫn đẹp. Bạn có một phép đo chính xác quanh một giá trị sai.

B. Bundle adjustment như một dụng cụ đo

Lối thoát, và nó là câu trả lời cho Q5: dùng chính camera làm dụng cụ đo.

Quy trình: chụp K ảnh của constellation từ nhiều vị trí và nhiều góc khác nhau (bao gồm cả những góc rất chếch — điều này quan trọng, vì lý do giống lý do ở A/01 mục 5 về điểm chính). Trong mỗi ảnh, phát hiện mọi tag nhìn thấy và lấy tọa độ góc. Rồi giải bài toán tối ưu với ẩn là: K pose camera và tọa độ 3D của mọi góc tag trong hệ dock. Hàm mục tiêu là tổng sai số tái chiếu trên toàn bộ [1].

Vì sao kết quả lại chính xác hơn thước cặp? Ba lý do.

Thứ nhất, *số lượng phép đo*. Với 30 ảnh và 4 tag, bạn có $30 \times 4 \times 4 = 480$ phép đo hai chiều, tức 960 phương trình, cho một số ẩn nhỏ hơn nhiều. Thông tin dư thừa lớn, và sai số ngẫu nhiên giảm theo căn bậc hai của nó.

Thứ hai, *bạn đo đúng thứ cần đo*. Thước cặp đo mép tấm nhôm; PnP dùng góc của tag in trên đó. Giữa hai thứ có sai lệch dán và sai lệch in mà thước cặp không thấy. BA đo chính xác đại lượng mà pipeline sẽ dùng.

Thứ ba, *nó đo trong hệ tọa độ đúng*. Kết quả BA là tọa độ của mọi góc trong một hệ chung, đã bao gồm mọi sai lệch góc quay của từng tag — thứ mà đo khoảng cách bằng thước không cho.

Có một điều kiện tiên quyết cần nói rõ: BA chỉ đo được constellation *sai khác một hệ số tỉ lệ* nếu chỉ dùng ảnh. Đây là tự do gauge về tỉ lệ, và nó phải được cố định bằng một thông tin bên ngoài: kích thước thật của một tag (đo bằng thước, và đây là chỗ duy nhất thước cặp cần thiết), hoặc một khoảng cách đã biết chính xác. Sai số của phép đo neo ấy trở thành sai số tỉ lệ của toàn bộ constellation, nên hãy đo nó cho cẩn thận — đo cạnh của tag lớn nhất chứ không phải tag nhỏ nhất.

CẠM BÃY

Ba sai lầm khi chạy BA để đo constellation.

Một, chỉ chụp từ các góc gần vuông góc. Với tư thế fronto-parallel, ambiguity nặng và thông tin về cấu trúc ba chiều nghèo, nên BA hội tụ kém và kết quả không đáng tin hơn CAD bao nhiêu. Phải có ảnh ở góc chếch mạnh — đây là cùng một yêu cầu, vì cùng một lý do, như yêu cầu về tư thế trong hiệu chuẩn intrinsic ở A/01 mục 5.

Hai, dùng intrinsic chưa hiệu chuẩn tốt. BA giải cấu trúc với K cho trước; nếu K sai thì cấu trúc thu được sai một cách có hệ thống, thường là sai tỉ lệ hoặc sai theo kiểu “cong” toàn cục. Thứ tự bắt buộc là: hiệu chuẩn intrinsic trước (B/12), rồi mới đo constellation.

Ba, không kiểm hiệp phương sai của kết quả. BA trả về không chỉ tọa độ mà cả bất định của chúng. Nếu một tag có bất định 2 mm thì bạn chưa đo được nó tốt hơn CAD, và cần chụp thêm ảnh phủ tag ấy từ nhiều góc hơn. Bỏ qua bước này là tự lừa mình: bạn đã thay một con số không biết sai bao nhiêu bằng một con số khác cũng không biết sai bao nhiêu.

1.6 Nguyên tắc năm: nested multi-scale

Mục này trả lời Q3, và nó là mục về ràng buộc vận hành.

Tag có kích thước ảnh $w = fW/Z$, và nó phải nằm trong một dải chấp nhận được. Cận dưới $w_{\min}$ đến từ hai nguồn: decode cần chừng 3–5 px mỗi ô bit (A/06 mục 4), và điều kiện số của bài toán xấu đi khi tag nhỏ (A/02 mục 4). Cận trên $w_{\max}$ đến từ việc tag phải *lọt trong khung hình* — và với nhiều tag thì phải lọt cả constellation, đây mới là ràng buộc thật.

Với dải khoảng cách $[Z_{\min}, Z_{\max}]$ và một cỡ tag duy nhất W , tỉ số kích thước ảnh giữa hai đầu dải là

$$\frac{w(Z_{\min})}{w(Z_{\max})} = \frac{Z_{\max}}{Z_{\min}}.$$

Với Z từ 3 m xuống 0,4 m, tỉ số là 7,5 — đúng con số trong Q3. Nếu ở xa tag chỉ vừa đủ lớn để decode ($w = 30$ px) thì ở gần nó chiếm $30 \times 7,5 = 225$ px; với một constellation gồm nhiều tag trải rộng, cả cụm sẽ tràn khỏi khung hình rất trước khi tới 0,4 m. Ngược lại, nếu chọn tag đủ nhỏ để cả cụm lọt khung ở 0,4 m, thì ở 3 m nó quá nhỏ để decode.

Đây là lý do *một cỡ tag không đủ cho một dải khoảng cách rộng*, và nó là vấn đề ở cả hai đầu chứ không chỉ một.

Giải pháp là nhiều cỡ. Nếu một cỡ tag phủ được tỉ số khoảng cách $r = w_{\max}/w_{\min}$, thì số cỡ cần thiết là

$$n = \left\lceil \frac{\ln(Z_{\max}/Z_{\min})}{\ln r} \right\rceil.$$

Ví dụ: nếu dải chấp nhận được của kích thước ảnh là 40–200 px thì $r = 5$; với $Z_{\max}/Z_{\min} = 7,5$, ta cần $n = \lceil \ln 7,5 / \ln 5 \rceil = \lceil 1,25 \rceil = 2$ cỡ. Hai cỡ tag, với tỉ lệ kích thước giữa chúng khoảng $\sqrt{7,5} = 2,7$ — chẳng hạn tag 200 mm và tag 75 mm.

Cách bố trí: tag lớn ở ngoại vi (chúng sẽ tràn khỏi khung khi tới gần, và điều đó không sao vì lúc ấy tag nhỏ đã đủ lớn), tag nhỏ ở trung tâm và chúng là cụm làm việc ở cự ly gần. Quan trọng: *cụm tag nhỏ tự nó cũng phải là một constellation tốt* — có baseline, không đồng phẳng — vì ở cự ly gần thì chỉ còn chúng.

1.7 Trọng số và loại ngoại lai

Mục này trả lời Q4 và nó nối với A/03 mục 6 và 7.

Trong một constellation, các tag *không* bình đẳng, và sự bất bình đẳng lớn hơn nhiều so với trong một target đơn. Một tag ở gần và nhìn thẳng có góc chính xác hơn hẳn một tag ở xa và nghiêng mạnh; một tag nghiêng 30° bằng nêm có cạnh bị nén nên định vị kém hơn; một tag ở rìa ảnh chịu méo còn sót lớn hơn. Cho chúng trọng số như nhau là để những tag kém nhất kéo nghiệm.

Quy tắc trọng số tối thiểu nên có: $\Sigma_i \propto 1/w_m^2$ với w_m là kích thước ảnh của tag chứa góc i . Nó đơn giản, nó nắm được nguồn biến thiên lớn nhất, và nó đã tốt hơn nhiều so với trọng số đều. Nếu detector của bạn cung cấp hiệp phương sai cho từng góc (B/08) thì dùng nó — tốt hơn nữa vì nó dị hướng.

Về loại ngoại lai, có một điểm đặc thù của constellation đáng nêu. Với Q4 — một tag bị bắn nên góc lệch 2 px — cơ chế lan truyền là thế này: PnP hợp nhất cực tiểu tổng bình phương, nên một góc lệch 2 px đóng góp $(2/0,2)^2 = 100$ lần một góc bình thường, và nó kéo pose về phía nó. Với 4 tag (16 góc), một góc xấu chiếm một phần đáng kể tổng hàm mục tiêu.

Ba cơ chế phát hiện, và nên có cả ba.

Cổng chi-square trên phần dư từng góc. Sau khi hội tụ, loại góc có phần dư chuẩn hoá vượt phân vị 99%, giải lại. Bắt được góc lệch rõ.

Kiểm nhất quán liên tag. Giải PnP riêng cho từng tag, so các pose. Nếu pose từ tag A và tag B lệch quá ngưỡng thì nghi ngờ. Điều làm phép thử này giá trị: nó bắt được loại lỗi mà cổng chi-square

bỏ sót — trường hợp *cả bốn góc* của một tag cùng lệch theo một hướng, chẳng hạn khi tag bị dán lệch vị trí so với mô hình. Bốn góc cùng lệch thì phần dư của chúng được hấp thụ vào pose và không góc nào nổi bật, nhưng pose từ tag ấy sẽ lệch so với pose từ tag khác.

RANSAC ở *mức tag*. Khi số tag đủ lớn (từ bốn trở lên), chọn ngẫu nhiên tập con hai tag, giải, đếm tag phù hợp [2]. Bắt được trường hợp một tag bị lắp hằn sai chỗ.

CẦN NHỚ

Ba cơ chế phát hiện bắt ba loại lỗi khác nhau, và không cái nào thay thế được cái nào.

Cổng chi-square bắt *một góc lệch* — che khuất một phần, một góc bị loá.

Kiểm nhất quán liên tag bắt *cả một tag lệch* — dán lệch, mô hình sai, hoặc tag đang ở nghiệm ambiguity khác. Đây là lỗi mà chi-square bỏ sót, và nó là lỗi phổ biến nhất trong constellation.

RANSAC bắt *một tag sai hằn* — lắp nhầm vị trí, nhầm ID.

Chi phí của cả ba cộng lại vẫn không đáng kể so với detector, nên không có lý do gì chỉ dùng một.

1.8 Tổng hợp: một bản thiết kế mẫu

Gom năm nguyên tắc thành một bố trí cụ thể, để có một điểm xuất phát chứ không phải một danh sách.

Giả sử bài toán: robot tiếp cận từ 3 m xuống 0,4 m, ngân sách 5 mm / 0,5°, camera $f = 600$ px với khung hình 1280×960.

Hai cỡ tag theo mục 6: bốn tag 200 mm ở ngoại vi cho giai đoạn xa, bốn tag 75 mm ở trung tâm cho giai đoạn gần.

Baseline ưu tiên ngang theo mục 3: cụm lớn trải 600 mm ngang × 250 mm đứng; cụm nhỏ trải 220 mm ngang × 90 mm đứng.

Phá đồng phẳng theo mục 4: mỗi tag trên một nêm nghiêng 20°, hướng nghiêng luân phiên — tag trái nghiêng sang trái, tag phải nghiêng sang phải, tag trên nghiêng lên, tag dưới nghiêng xuống. Bố trí luân phiên này cho độ lệch khỏi mặt phẳng lớn hơn so với nghiêng cùng chiều, và nó cũng đảm bảo mỗi tag có góc nghiêng khác không so với trục quang ở tư thế tiếp cận.

Đo bằng BA theo mục 5, sau khi lắp và trước khi vận hành, với ít nhất 30 ảnh từ nhiều góc, neo tỉ lệ bằng cạnh đo được của một tag 200 mm.

Trọng số và ba cơ chế phát hiện theo mục 7.

Kiểm nhanh bằng công thức của A/05: ở $Z = 0,5$ m với $B = 0,22$ m (cụm nhỏ), $\delta\theta = 0,5 \times 0,2 / (600 \times 0,22) = 7,6 \times 10^{-4}$ rad = 0,043°. Dư so với 0,5° — nhưng nhớ rằng dòng lớn hơn nhiều là sai số constellation ở mục 5, và đó là lý do BA bắt buộc.

(Bản thiết kế này là ví dụ minh hoạ do tôi dựng từ các nguyên tắc của chương, không phải một thiết kế đã được chế tạo và đo. Nó nên được kiểm bằng Monte Carlo ở C/16 trước khi đưa vào gia công.)

1.9 Giới hạn và chế độ hồng

Constellation lớn hơn có thể bị che khuất. Trải rộng tốt cho hình học nhưng khi robot tới rất gần, một phần constellation có thể ra khỏi khung hình hoặc bị chính thân robot che. Đây là lý do

cụm nhỏ ở trung tâm trong thiết kế mẫu, và nó là một ràng buộc thật phải kiểm bằng cách mô phỏng trường nhìn dọc toàn bộ quỹ đạo tiếp cận.

Nêm nghiêng làm tag khó nhìn từ một số hướng. Tag nghiêng 30° sang trái sẽ khó đọc khi robot tiếp cận từ bên phải. Với đường tiếp cận cố định thì không sao; với robot có thể tới từ nhiều hướng thì phải cân nhắc, và góc nghiêng nên nhỏ hơn.

Constellation cứng là một giả định. Toàn bộ chương giả định các tag gắn cứng với nhau và mô hình không đổi. Rung động, va chạm, giãn nở nhiệt đều vi phạm. Cách phòng: chỉ báo sai số tái chiếu ở mục 2 chạy định kỳ, và đo lại BA sau bất kỳ va chạm nào.

BA có thể hội tụ về nghiệm sai. Như mọi bài toán tối ưu phi tuyến, BA cần khởi tạo tốt. CAD là khởi tạo tốt — đây là chỗ CAD vẫn hữu ích, dù không đủ chính xác để dùng trực tiếp. Và phải kiểm hiệp phương sai của kết quả, như đã cảnh báo ở mục 5.

Các con số khuyến nghị chưa được kiểm. Góc nghiêng $15\text{--}30^\circ$, tỉ lệ baseline đứng trên ngang một phần ba, độ lệch mặt phẳng 20% kích thước tag — cả ba là khuyến nghị của tôi dựa trên cơ chế, không phải kết quả tối ưu hoá. Chúng là điểm xuất phát hợp lý và chúng nên được kiểm bằng mô phỏng cho cấu hình cụ thể.

1.10 Thực hành

Năm việc, theo thứ tự thời gian của một dự án.

Một, trước khi vẽ bản vẽ: chạy Monte Carlo cho hai ba phương án bố trí, dùng script của A/05 mở rộng. Một buổi chiều ở đây quyết định phần lớn kết quả.

Hai, khi vẽ bản vẽ: đưa nêm nghiêng vào ngay. Đây là chi tiết dễ quên nhất và đắt nhất để thêm sau.

Ba, sau khi lắp: chạy BA đo constellation, kiểm hiệp phương sai kết quả, và lưu toạ độ đo được thành tệp cấu hình — không dùng số từ CAD.

Bốn, trong vận hành: bật cả ba cơ chế phát hiện ở mục 7, và ghi chỉ báo sai số tái chiếu hợp nhất so với từng tag vào log.

Năm, định kỳ: chạy lại chỉ báo ấy và so với giá trị lúc lắp đặt. Tăng đáng kể nghĩa là constellation đã thay đổi — va chạm, vít lỏng, hoặc trôi nhiệt.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba kết quả Phần A hội tụ về chương này	$\delta\theta \approx Z\sigma/(fB)$ (A/05); không đồng phẳng khử ambiguity (A/04); PnP hợp nhất thắng trung bình pose (A/03). Cả ba đều là quyết định <i>cơ khí</i> , không phải phần mềm.
••• Điều kiện của PnP hợp nhất	Toạ độ 3D constellation phải đúng. Hợp nhất chuyển sai số mô hình <i>thắng</i> vào sai số pose. Nên quyết định dùng hợp nhất và quyết định đo bằng BA là một cặp không tách rời.
••• Chỉ báo constellation sai	So sai số tái chiếu hợp nhất với sai số khi giải từng tag riêng. Hợp nhất lớn hơn hẳn $\Rightarrow$ mô hình sai. Miễn phí, và nên chạy định kỳ chứ không chỉ lúc lắp.
••• Baseline có hướng	Yaw cần baseline <i>ngang</i> (quan trọng nhất cho docking); pitch cần baseline <i>đứng</i> ; roll gần như luôn đạt. Ưu tiên ngang, nhưng đừng bỏ hẳn đứng — ma trận gần suy biến làm sai số rò rỉ sang hướng khác.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao nệm nghiêng là chi tiết đáng giá nhất	Nó giải quyết <i>hai</i> bài toán độc lập bằng một chi tiết: ambiguity ($A/\theta 4$) và độ nhạy $\sin \theta \rightarrow 0$ ($A/\theta 5$). Chi phí: một khối nhôm vát góc và bốn vít.
●●● Sai số CAD 1,4 mm ăn bao nhiêu ngân sách	Trên baseline 400 mm: $\arctan(1,4/400) = 0,2^\circ$, tức 40% ngân sách góc. Và nó là <i>hệ thống</i> — cộng thẳng, không chia $\sqrt{N}$, không hiện ra ở chỉ số nào.
●●● Vì sao BA chính xác hơn thước cặp	Ba lý do: số phép đo lớn (30 ảnh $\times$ 4 tag $\times$ 4 góc = 960 phương trình); nó đo <i>đúng thứ pipeline dùng</i> (góc tag in, không phải mép nhôm); và nó cho toạ độ trong <i>một hệ chung</i> gồm cả sai lệch góc từng tag.
●●● Tự do gauge về tỉ lệ trong BA	Ảnh đơn thuần chỉ xác định constellation sai khác một hệ số tỉ lệ. Phải neo bằng một khoảng cách đo được — và đo cạnh tag <i>lớn nhất</i> , vì sai số của phép neo trở thành sai số tỉ lệ toàn cục.
●● Ba sai lầm khi chạy BA	Chỉ chụp góc gần vuông (thông tin nghèo, hội tụ kém); dùng intrinsic chưa hiệu chuẩn (cấu trúc sai có hệ thống); không kiểm hiệp phương sai kết quả (thay một con số chưa biết sai bằng một con số khác chưa biết sai).
●●● Số cỡ tag cần thiết	$n = \lceil \ln(Z_{\max}/Z_{\min}) / \ln r \rceil$ với $r = w_{\max}/w_{\min}$. Ví dụ: dải 3 m–0,4 m và $r = 5 \Rightarrow 2$ cỡ, tỉ lệ kích thước $\sqrt{7,5} \approx 2,7$.
●● Bố trí nested multi-scale	Tag lớn ở ngoại vi (tràn khỏi khung khi gần, không sao); tag nhỏ ở trung tâm. Quan trọng: <i>cụm nhỏ tự nó cũng phải là constellation tốt</i> — có baseline, không đồng phẳng — vì ở gần chỉ còn nó.
●●● Ba cơ chế phát hiện, ba loại lỗi	Chi-square bắt <i>một góc</i> lệch; kiểm nhất quán liên tag bắt <i>cả một tag</i> lệch (chi-square bỏ sót vì bốn góc cùng lệch được hấp thụ vào pose); RANSAC bắt tag lắp sai hần.
●● Quy tắc trọng số tối thiểu	$\Sigma_i \propto 1/w_m^2$ theo kích thước ảnh của tag chứa góc đó. Đơn giản, nắm được nguồn biến thiên lớn nhất, và tốt hơn nhiều so với trọng số đều.
● Các con số khuyến nghị chưa kiểm	Nghiêng 15–30°; baseline đứng $\geq 1/3$ ngang; lệch mặt phẳng $\geq 20\%$ kích thước tag. Đây là khuyến nghị của tôi dựa trên cơ chế, chưa qua cửa kiểm chứng — kiểm bằng Monte Carlo.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ArUco Board / ChArUco Board	Coi cả bảng là một target, giải một PnP trên toàn bộ — đúng nguyên tắc mục 2	Nhưng board phẳng, nên vẫn có ambiguity; dùng làm nền rồi thêm nệm nghiêng
tagslam [3]	Map nhiều tag trong môi trường bằng factor graph, ước lượng cả vị trí tag	Chính là BA của mục 5 ở quy mô phòng; mức tin C, dùng làm gợi ý kiến trúc
COLMAP [4]	Photogrammetry tổng quát; dùng đo constellation khi không muốn tự viết BA	Đường tắt thực dụng cho mục 5 — xem D/18
Ceres [5], GTSAM	Viết BA riêng với ràng buộc phù hợp (tag cứng, góc tag cố định tương đối)	Cách đúng nếu muốn tận dụng cấu trúc cứng của từng tag
Trắc lượng ảnh công nghiệp [1]	Truyền thống lâu đời của việc đo lại mốc thay vì đặt mốc chính xác	Nguồn của toàn bộ ý tưởng ở mục 5

MINI PROJECT · So bốn phương án bố trí bằng Monte Carlo trước khi gia công · 2–3 ngày

Mục tiêu	biến quyết định “đặt tag ở đâu” từ một phán đoán thành một bảng số, và có một bản vẽ có cơ sở trước khi tốn tiền gia công.
Dữ liệu	mô phỏng thuần, mở rộng script của A/05 và A/04.
Các bước	đựng bốn phương án: (A) một tag lớn ở giữa; (B) bốn tag đồng phẳng ở bốn góc; (C) bốn tag với nêm nghiêng 20° luân phiên; (D) phương án C cộng thêm cụm nhỏ ở trung tâm. Với mỗi phương án, mô phỏng toàn bộ quỹ đạo tiếp cận từ $Z_{\max}$ tới $Z_{\min}$ với các giá trị lệch ngang và lệch góc ban đầu khác nhau; ở mỗi điểm, chiếu mọi tag nhìn thấy (kiểm cả điều kiện lọt khung hình và điều kiện kích thước tối thiểu), thêm nhiễu, chạy PnP hợp nhất, ghi sai số vị trí và góc <i>cùng với</i> tỉ số ambiguity ρ ; 2000 lần cho mỗi điểm. Sau đó phần thứ hai: thêm sai số constellation e vào mô hình và quét e từ 0 tới 3 mm.
Xong khi	bạn có bốn đường sai số góc theo Z đặt cạnh nhau, và một bảng nói mỗi phương án đạt ngưỡng ở dải khoảng cách nào. Kiểm tra chéo bắt buộc: phương án B phải cho $\rho \approx 1$ ở tư thế gần vuông góc trong khi phương án C thì không — nếu cả hai đều cho ρ lớn thì mô phỏng ambiguity của bạn có lỗi, vì A/04 nói rõ B phải có vấn đề. Và từ phần hai bạn có ngưỡng e cho phép, tức yêu cầu độ chính xác của BA.
Nối với	C/15 nhận bảng này làm dòng hình học của ngân sách; B/12 nhận ngưỡng e làm yêu cầu cho bước hiệu chuẩn constellation.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Phương án B, và chênh lệch lớn vì hai lý do độc lập cộng lại. Lý do thứ nhất: B có baseline 400 mm giống A, nên về mặt độ nhạy góc hai phương án tương đương — việc A có bốn tag thay vì hai chỉ giảm nhiễu theo $\sqrt{2}$, một cải thiện nhỏ. Lý do thứ hai, và đây mới là phần lớn: A vẫn *đồng phẳng*, nên nó vẫn có hai nghiệm và vẫn chịu độ nhạy $\sin \theta \rightarrow 0$ ở tư thế tiếp cận thẳng trục; B thì không, vì hai nêm nghiêng ngược chiều nhau tạo độ lệch khỏi mặt phẳng lớn và đồng thời cho mỗi tag một góc nghiêng khác không so với trục quang. Nói ngắn: hai tag nghiêng thẳng bốn tag phẳng, và bài học là số lượng tag gần như không quan trọng so với hình học của chúng.
- A2. Nó ăn khoảng 40% ngân sách góc: $\arctan(1,4/400) = 0,2^\circ$ so với ngân sách $0,5^\circ$. Và nó ăn theo cách tệ nhất — đây là sai số *hệ thống*, nên nó cộng thẳng chứ không cộng bình phương với các nguồn khác, và nó không giảm chút nào khi lấy trung bình một nghìn khung. Nó không hiện ra ở chỉ số nào vì PnP hợp nhất sẽ làm đúng việc nó được giao: tìm pose tốt nhất giải thích dữ liệu *dưới mô hình đã cho*. Mô hình sai thì pose sai, sai số tái chiếu tăng một chút nhưng vẫn nhỏ, và hiệp phương sai báo về vẫn đẹp vì công thức hiệp phương sai giả định mô hình đúng (A/05 mục 2). Cách duy nhất phát hiện là so với một nguồn thông tin độc lập — chẳng hạn so sai số tái chiếu hợp nhất với từng tag riêng, hoặc đo lại bằng BA.
- A3. Ở đầu xa, tag quá nhỏ: dưới ngưỡng 3–5 px mỗi ô bit thì decode không tin cậy, và điều kiện số của bài toán xấu đi nhanh hơn $1/w$. Ở đầu gần, vấn đề ngược lại và ít người lường trước: constellation trải rộng sẽ *tràn khỏi khung hình*, nên bạn mất đúng các tag ngoại vi vốn cung cấp baseline, đúng vào lúc cần chính xác nhất. Với dải 3 m–0,4 m, tỉ số 7,5. Nếu dải kích thước ảnh chấp nhận được là 40–200 px (tỉ số 5) thì cần $\lceil \ln 7,5 / \ln 5 \rceil = 2$ cỡ tag, với tỉ lệ kích thước giữa chúng khoảng 2,7 lần. Điều quan trọng thường bị bỏ qua: cụm tag nhỏ phải tự nó là một constellation tốt — có baseline, không đồng phẳng — vì ở cự ly gần thì chỉ còn nó làm việc.
- A4. Lan truyền theo cơ chế bình phương: một góc lệch 2 px đóng góp $(2/0,2)^2 = 100$ lần một góc bình thường vào hàm mục tiêu, nên nó kéo pose về phía nó, và với 16 góc thì nó chiếm phần đáng kể tổng. Ba cơ chế phát hiện, bắt ba loại lỗi khác nhau: cộng chi-square trên phần dư từng góc bắt được trường hợp *một góc lệch rõ* (đúng tình huống này); kiểm nhất quán liên tag — giải PnP riêng từng tag rồi so pose — bắt được trường hợp *cả bốn góc của một tag cùng lệch*, thứ mà chi-square bỏ sót vì lệch đồng đều được hấp thụ vào pose; và RANSAC ở mức tag bắt được tag lắp sai hẳn. Cả ba nên bật cùng lúc vì chi phí không đáng kể.
- A5. Bundle adjustment: chụp vài chục ảnh constellation từ nhiều vị trí và góc, rồi giải đồng thời cả pose các khung lẫn tọa độ 3D của mọi góc tag. Nó chính xác hơn thước cặp vì ba lý do. Số lượng: 30 ảnh $\times$ 4 tag $\times$ 4 góc cho 960 phương trình cho một số ẩn nhỏ hơn nhiều, nên sai số ngẫu nhiên bị dìm. Đúng đại lượng: thước cặp đo mép tấm nhôm, còn pipeline dùng góc của tag *in* trên đó — giữa hai thứ có sai lệch in và sai lệch dán mà thước không thấy. Và đúng hệ tọa

độ: BA cho tọa độ mọi góc trong một hệ chung, đã bao gồm sai lệch góc quay của từng tag, thứ mà đo khoảng cách không cho. Điều kiện duy nhất: ảnh không xác định được tỉ lệ, nên phải neo bằng một khoảng cách đo được — và đó là chỗ duy nhất thước cặp còn cần thiết, nên hãy đo cạnh của tag lớn nhất.

NÓI LẠI THẬT ĐƠN GIẢN

Nếu chỉ có một chương trong cả tài liệu này được đọc bởi người thiết kế cơ khí, thì nên là chương này — vì gần như mọi thứ trong đây đều được quyết định bằng nhôm và vít chứ không bằng code. Có ba việc phải làm với mấy tấm tag, và chúng độc lập nhau.

Việc thứ nhất: *đặt chúng xa nhau*. Muốn biết một vật đang quay bao nhiêu, hãy nhìn hai điểm trên nó ở xa nhau — cùng một góc quay làm chúng dịch ngược chiều nhau nhiều hơn, nên dễ đo hơn. Hai tag cách nhau nửa mét cho phép đo góc tốt gấp năm lần hai tag cách nhau mười phân, và cái giá phải trả chỉ là một tấm nhôm rộng hơn. Với robot cập dock thì hướng quan trọng nhất là trái-phải, nên hãy trải rộng theo chiều ngang trước.

Việc thứ hai: *đừng để chúng phẳng lì*. Kê một cái nêm dưới mỗi tag cho nó nghiêng đi hai mươi độ, mỗi cái nghiêng một hướng. Lý do nghe lạ nhưng rất thật: một tấm phẳng nhìn từ phía trước thì có hai cách nghiêng cho cùng một hình ảnh, và máy không phân biệt được, nên nó cứ nhảy qua nhảy lại giữa hai câu trả lời. Kê nghiêng thì hết. Cái nêm ấy tốn vài chục nghìn và nó giải quyết một vấn đề mà hàng tuần viết code không giải quyết được.

Việc thứ ba, và đây là việc hay bị bỏ qua nhất: *đo lại xem chúng thực sự ở đâu*. Bản vẽ nói hai tag cách nhau bốn trăm milimét, nhưng sau khi cắt, khoan, in, dán và lắp thì thực tế có thể là ba trăm chín mươi tám phẩy sáu. Một phẩy bốn milimét nghe chẳng đáng gì, nhưng nó ăn gần một nửa cái ngân sách góc mà bạn có — và tệ nhất là nó không lộ ra ở đâu cả. Máy vẫn chạy, con số vẫn đẹp, và mọi phép đo đều lệch đi cùng một chút.

Cách đo lại thì bất ngờ đơn giản: dùng chính cái camera. Chụp mấy chục tấm ảnh của cả cụm từ nhiều góc, rồi để máy tính giải ngược ra xem mỗi tag thật sự nằm đâu. Cách này chính xác hơn thước cặp, vì nó dùng cả nghìn phép đo thay vì vài phép, và vì nó đo đúng cái thứ mà máy sẽ dùng sau này — các góc của tag in, chứ không phải mép tấm nhôm.

Cuối cùng, một chuyện về khoảng cách. Cái tag trông to hay nhỏ tùy robot đứng gần hay xa. Ở xa ba mét nó nhỏ quá không đọc được; ở gần bốn tấc thì cả cụm tràn ra ngoài khung hình. Không có cỡ nào vừa cho cả hai. Nên cách làm là dán hai cỡ: mấy cái to ở ngoài rìa cho lúc còn xa, mấy cái nhỏ ở giữa cho lúc đã gần — và nhớ rằng mấy cái nhỏ ở giữa cũng phải xa nhau và cũng phải kê nghiêng, vì lúc cập vào thì chỉ còn chúng nó.

Chỗ không rút gọn được: nghiêng bao nhiêu độ là đủ, và xa nhau bao nhiêu là đủ — hai con số ấy thì không có công thức. Chúng phụ thuộc camera, khoảng cách, và độ chính xác cần đạt. May là chạy thử bằng mô phỏng thì rẻ, và một buổi chiều mô phỏng rẻ hơn nhiều so với một lô hàng gia công sai.

Một câu: *Xa nhau, kê nghiêng, và đo lại — ba việc làm bằng nhôm và vít, và chúng quyết định nhiều hơn mọi dòng code cộng lại.*

1.11 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *sai số constellation là dòng chi phối trong ngân sách của một hệ đã tối ưu tốt phần nhiều* — bị bác bỏ nếu trong một hệ thực, sau khi đo constellation bằng BA và cải thiện độ chính xác của nó xuống dưới 0,2 mm, sai số pose không cải thiện đáng kể. Khi ấy nút thắt nằm ở chỗ khác — nhiều khả năng là hand-eye (B/12) hoặc sai số bám của bộ điều khiển (B/13). (Tôi phát biểu dựa trên phép tính độ lớn ở mục 5, không dựa trên thống kê hệ thực tế.)

Mệnh đề thứ hai — *hai tag nghiêng thẳng bốn tag phẳng* — là mệnh đề dễ kiểm nhất và có hậu quả thiết kế lớn nhất. Nó bị bác bỏ nếu Monte Carlo cho thấy bốn tag đồng phẳng cho sai số góc tương đương hoặc tốt hơn ở tư thế tiếp cận. Nếu vậy thì hoặc mô phỏng ambiguity có lỗi, hoặc lập luận của A/04 về mức độ nghiêm trọng của ambiguity là quá mạnh.

Mệnh đề thứ ba — *BA cho constellation chính xác hơn CAD* — bị bác bỏ nếu hiệp phương sai mà BA trả về lớn hơn dung sai gia công. Điều này *có thể* xảy ra nếu số ảnh ít, góc nhìn nghèo, hoặc intrinsic chưa tốt — và đó chính là lý do mục 5 nhấn mạnh phải kiểm hiệp phương sai kết quả. Nói cách khác, mệnh đề này không phải một sự thật vô điều kiện mà là một điều kiện phải kiểm cho từng lần đo.

Cuối cùng, ba con số khuyến nghị ở mục 4 và 3 — nghiêng $15\text{--}30^\circ$, tỉ lệ baseline đứng trên ngang một phần ba, lệch mặt phẳng 20% kích thước tag — tôi đã đánh dấu rõ là chưa qua cửa kiểm chứng nào. Chúng có thể sai theo cả hai chiều (quá bảo thủ hoặc chưa đủ), và mini project là cách kiểm.

1.12 Kết nối

Chương này là nút giao của cây: mọi thứ trong Phần A chảy vào đây và nó chảy ra thành một bản vẽ.

Nối lên trên. A/05-lan-truyen-sai-so-pixel-sang-pose cấp công thức $\delta\theta \approx \delta x/B$, và mục 3 làm rõ rằng B là đại lượng có hướng — một chi tiết mà công thức vô hướng giấu đi. A/04-pose-ambiguity-target-phang cấp lý do cho mục 4, và mini project của chương ấy cấp phương pháp đo ngưỡng độ lệch mặt phẳng. A/03-pnp-va-uoc-luong-tu-the cấp mục 2 và mục 7 gần như nguyên vẹn; điều chương này thêm vào là *điều kiện* của PnP hợp nhất và hệ quả của nó với mục 5. A/02-homography-va-hinh-hoc-phang mục 4 cấp lý do định lượng cho ngưỡng kích thước tối thiểu ở mục 6.

Nối ngang trong Phần B. B/08-corner-refinement cấp Σ_i cho mục 7, và nó cũng cho biết σ thực tế của hệ — con số đi vào mọi phép tính baseline. B/12-calibration-cho-docking là chương anh em của mục 5: nó bàn hiệu chuẩn intrinsic và hand-eye, còn mục 5 ở đây bàn hiệu chuẩn constellation, và ba thứ ấy phải làm theo đúng thứ tự (intrinsic trước, constellation sau, hand-eye cuối). B/10-xu-ly-ambiguity là chương xử lý phần dư của mục 4: những gì phải làm ở tầng code khi hình học chưa khử hết ambiguity.

Nối xuống Phần C. C/15-ngan-sach-sai-so nhận từ mục 5 dòng lớn nhất của bảng ngân sách, và nhận từ mục 3 phần hình học. C/16-danh-gia-va-nghiem-thu là nơi mini project của chương này sống, và nó mở rộng mô phỏng ấy thành quy trình nghiệm thu đầy đủ.

Và một nối sang B/13-docking-control đáng nêu: chương điều khiển sẽ lập luận rằng teach-by-showing triệt tiêu các sai số hệ thống. Điều đó *bao gồm* sai số constellation — nên nếu bạn dùng teach-by-showing, yêu cầu về độ chính xác của mục 5 nổi lỏng đáng kể. Đây là một trong những chỗ mà hai chương xương sống của Phần B tương tác, và cách đọc đúng là: *làm cả hai thì dư dả, làm một trong hai thì vừa đủ, không làm cái nào thì không đạt*.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hai tag nghiêng thắng bốn tag đồng phẳng, và hai lợi ích độc lập nào mà nệm nghiêng cung cấp cùng lúc?
2. Baseline là đại lượng có hướng. Với robot cập dock, thành phần xoay nào quan trọng nhất, nó cần baseline theo hướng nào, và vì sao không nên bỏ hẵn hướng còn lại?
3. Sai số constellation 1,4 mm trên baseline 400 mm ăn bao nhiêu phần trăm ngân sách góc? Tính, rồi nói vì sao con số ấy không hiện ra trong bất kỳ chỉ số nào của pipeline.
4. Nêu ba lý do BA cho constellation chính xác hơn đo bằng thước cặp, và nêu một chỗ mà thước cặp vẫn không thay thế được.
5. Với dải khoảng cách $Z_{\max}/Z_{\min} = 10$ và dải kích thước ảnh chấp nhận được 40–240 px, cần mấy cỡ tag và tỉ lệ giữa chúng là bao nhiêu?

6. Ba cơ chế phát hiện ngoại lai bắt ba loại lỗi khác nhau. Nêu loại lỗi mà cổng chi-square *bỏ sót*, và giải thích vì sao nó bỏ sót.
7. PnP hợp nhất tốt hơn trung bình pose — nhưng chỉ dưới một điều kiện. Nêu điều kiện ấy, và nêu chỉ báo miễn phí cho biết điều kiện có được thoả hay không.

Nguồn

- [1] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., “Bundle Adjustment — A Modern Synthesis,” in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [2] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [3] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [4] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [5] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.